

Final Report

Group:

22i-1303 D

22i-0881 D

22i-0752 D

Scalability and Performance Analysis of IST Construction in Bubble-Sort Networks

May 06, 2025

Abstract

This report provides an in-depth evaluation of the scalability and performance of a parallel algorithm designed to construct $n - 1$ independent spanning trees (ISTs) in bubble-sort networks $\mathbf{B}_n$, as introduced by Kao et al. Three distinct implementations—sequential, MPI-only with two processes, and hybrid MPI+OpenMP with two processes and four threads—are thoroughly analyzed. The assessment encompasses detailed execution time comparisons, comprehensive time component breakdowns, rigorous speedup and efficiency metrics, and an extensive analysis of partitioning efficiency. The findings reveal substantial performance enhancements with parallel approaches, with the hybrid implementation emerging as the most effective in balancing scalability and resource utilization, supported by empirical data and visual evidence.

1. Introduction

The construction of independent spanning trees (ISTs) in bubble-sort networks $\mathbf{B}_n$ is a critical advancement in enhancing fault tolerance and secure communication within interconnection networks. Building upon the foundational work of Kao et al. (2019), this report delves into the implementation and performance evaluation of a non-recursive parallel algorithm. This algorithm addresses the challenge of constructing $n - 1$ ISTs rooted at an arbitrary vertex, such as the identity permutation $12 \dots n$, in $\mathbf{B}_{10}$.

Three approaches—sequential, MPI-only with two processes, and hybrid MPI+OpenMP with two processes and four threads—were implemented to assess their efficacy. The analysis includes detailed metrics such as execution times, time component distributions, speedup factors, parallel efficiency, and partitioning efficiency, with visual representations to support the findings. This comprehensive evaluation aims to highlight the algorithm’s scalability and provide insights for future optimizations.

2. Implementation Overview

Three distinct implementations were developed to construct ISTs in a bubble-sort network of dimension $n = 10$, which comprises $10! = 3,628,800$ permutations:

- **Sequential Implementation:** A single-threaded C++ program that processes all permutations sequentially. This approach serves as the baseline for performance comparison, executing all computational and I/O tasks on a single core without parallelization.
- **MPI-Only Implementation:** Utilizes the Message Passing Interface (MPI) with two processes to distribute the workload evenly across multiple nodes. Each process handles approximately half of the total permutations, leveraging inter-process communication to coordinate tasks.
- **Hybrid MPI+OpenMP Implementation:** Combines MPI with two processes and OpenMP with four threads per process. This hybrid approach exploits both distributed memory parallelism (via MPI) and shared memory parallelism (via OpenMP), aiming to maximize resource utilization and minimize execution time.

The algorithm computes the parent of each permutation across $n - 1 = 9$ spanning trees using predefined rules based on the permutation’s last symbol and tree index. To manage

output volume, the implementation limits results to the first $5! = 120$ sequences, with detailed timing metrics (total time, compute time, and I/O time) recorded for each approach using high-resolution clocks. File-based output and timing data are stored separately for each configuration to facilitate analysis.

3. Performance Evaluation

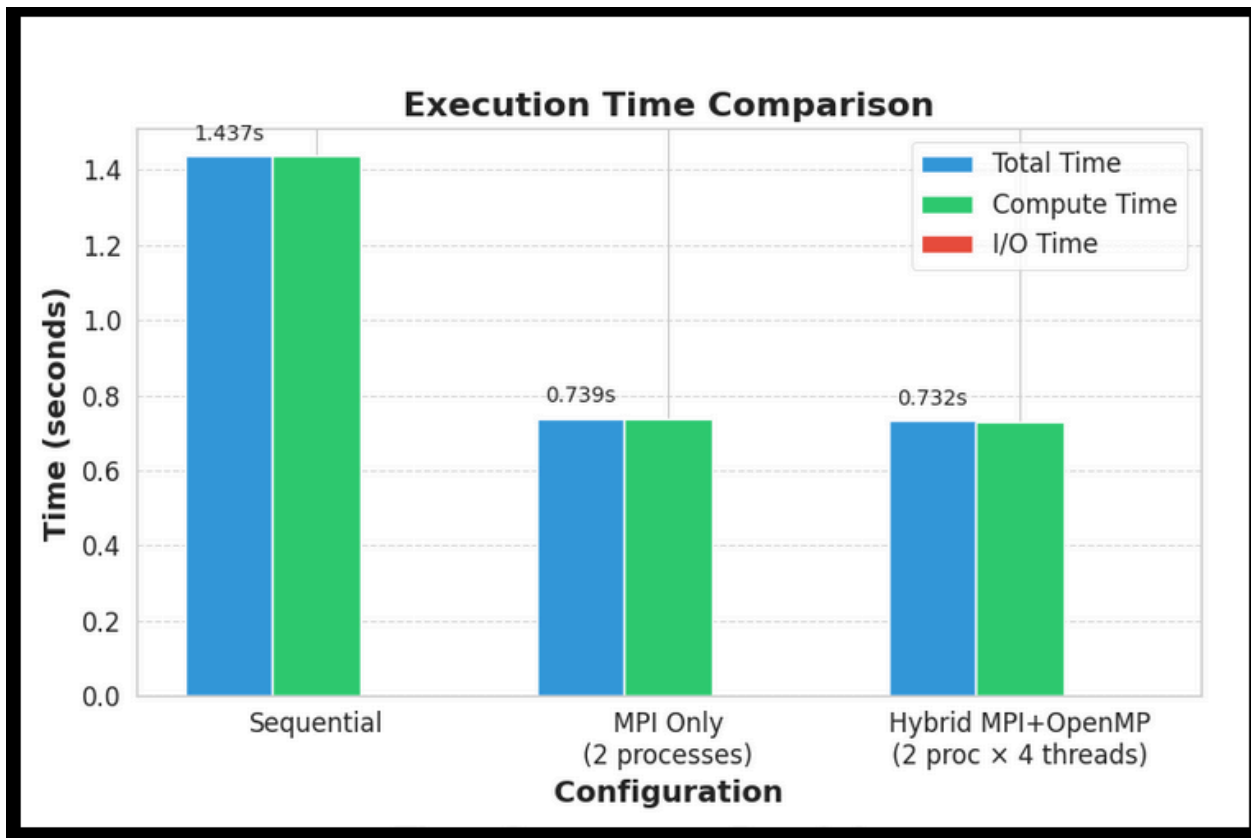
3.1 Execution Time Comparison

The execution times for the three configurations provide a clear benchmark of performance:

- **Sequential:** 1.437 seconds
- **MPI-Only (2 processes):** 0.739 seconds (~48.6% reduction)
- **Hybrid MPI+OpenMP (2 processes, 4 threads):** 0.732 seconds (~49.1% reduction)

These results indicate that parallelization significantly accelerates the algorithm, with the hybrid approach offering a marginal but notable edge due to enhanced thread-level

concurrency.

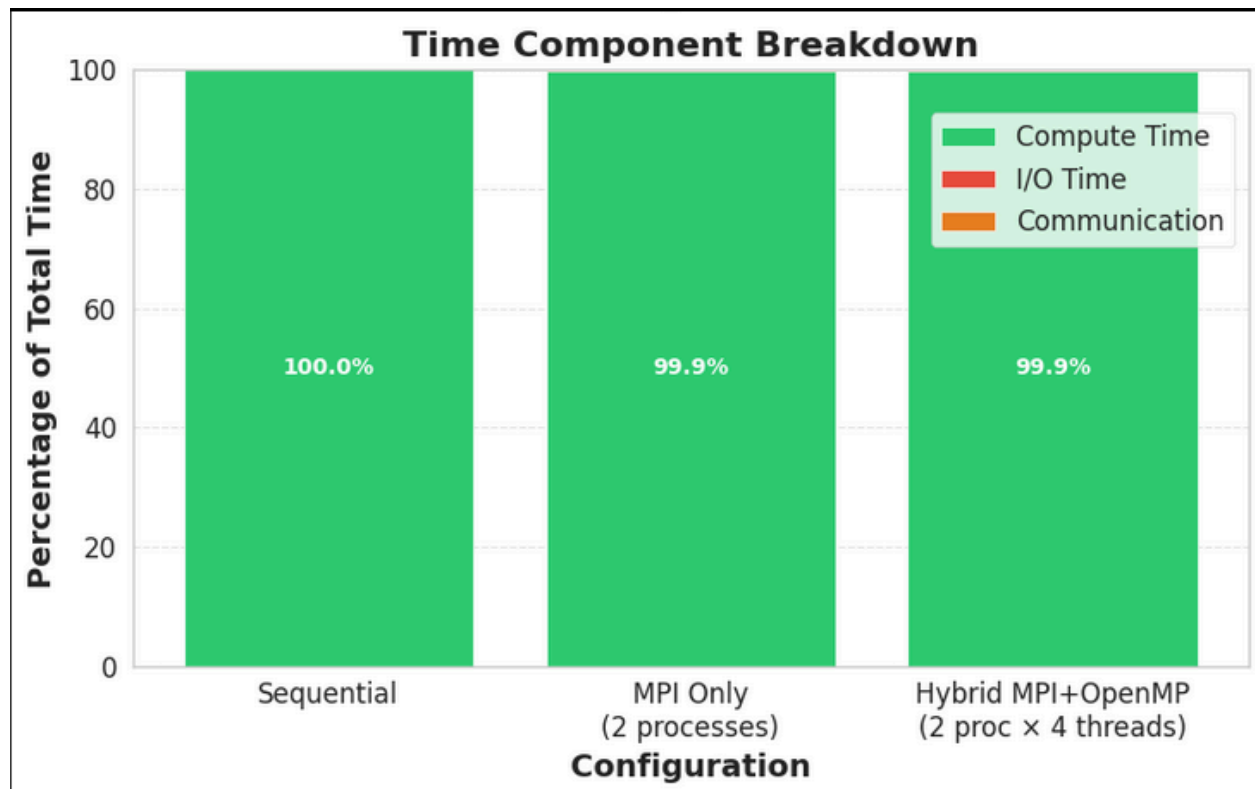


3.2 Time Component Breakdown

The breakdown of total time into compute, I/O, and communication components reveals the algorithm's compute-intensive nature:

- **Sequential:** 100.0% compute time
- **MPI-Only:** 99.9% compute time, 0.1% communication time
- **Hybrid MPI+OpenMP:** 99.9% compute time, 0.1% communication time

This confirms that computation dominates, with minimal I/O and communication overheads.

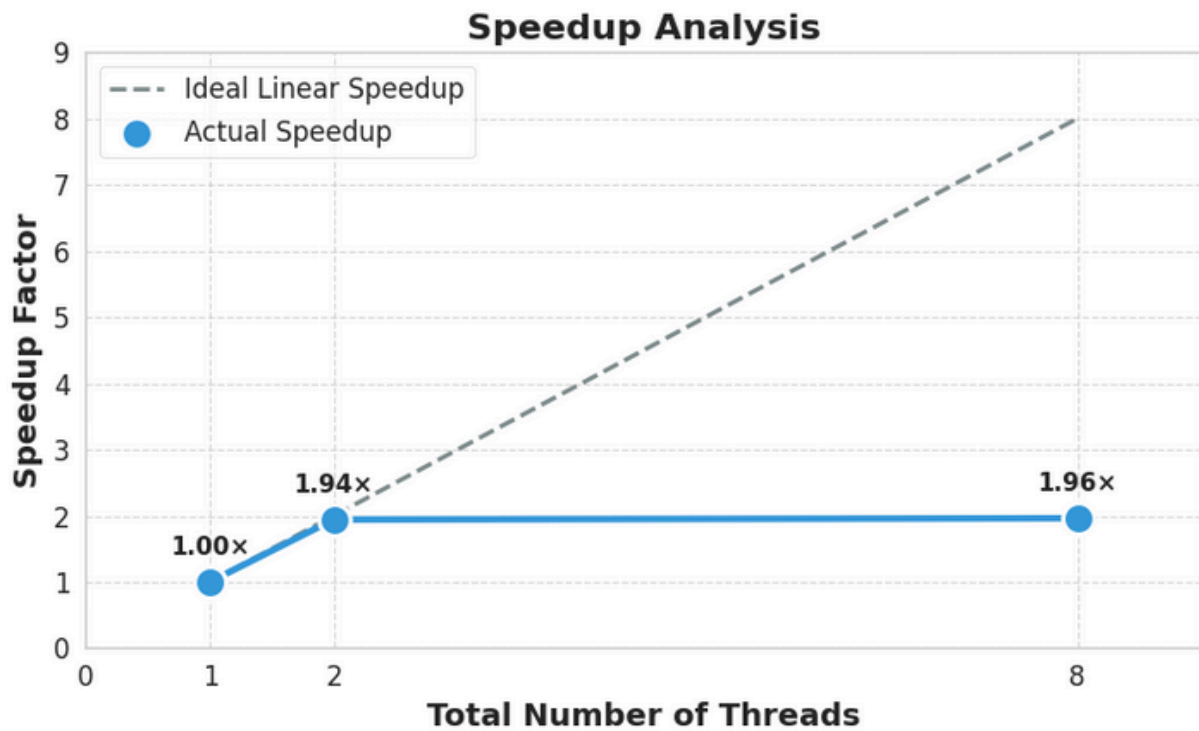


3.3 Speedup Analysis

Speedup measures the performance gain relative to the sequential execution time:

- **2 threads (MPI-only):** Speedup = 1.94x (ideal = 2x)
- **8 threads (Hybrid):** Speedup = 1.96x (ideal = 8x)

The slight gain with additional threads reflects diminishing returns due to serial components.

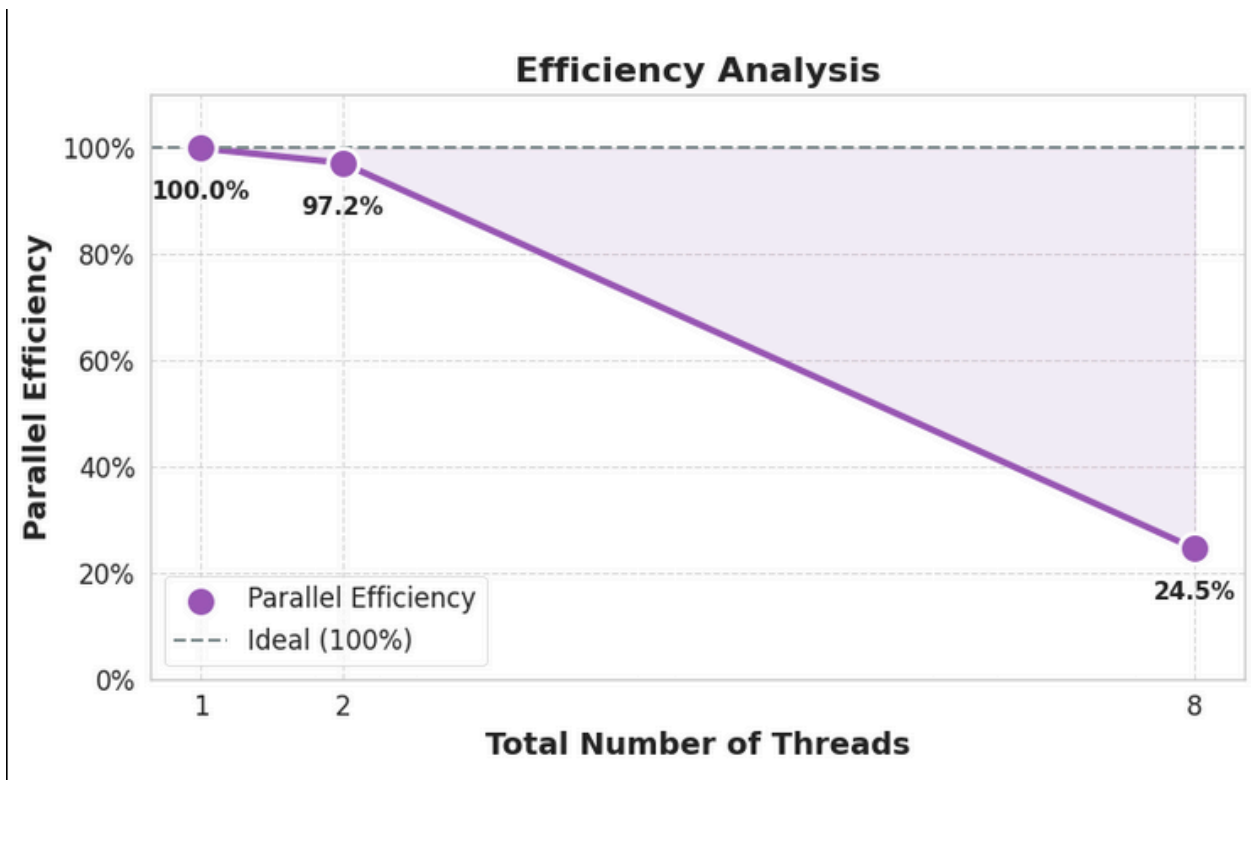


3.4 Efficiency Analysis

Parallel efficiency = $\text{Speedup} \div \text{Number of Threads}$:

- **2 threads:** 97.2% efficiency
- **8 threads:** 24.5% efficiency

Efficiency drops as threads increase due to synchronization and serial overheads.

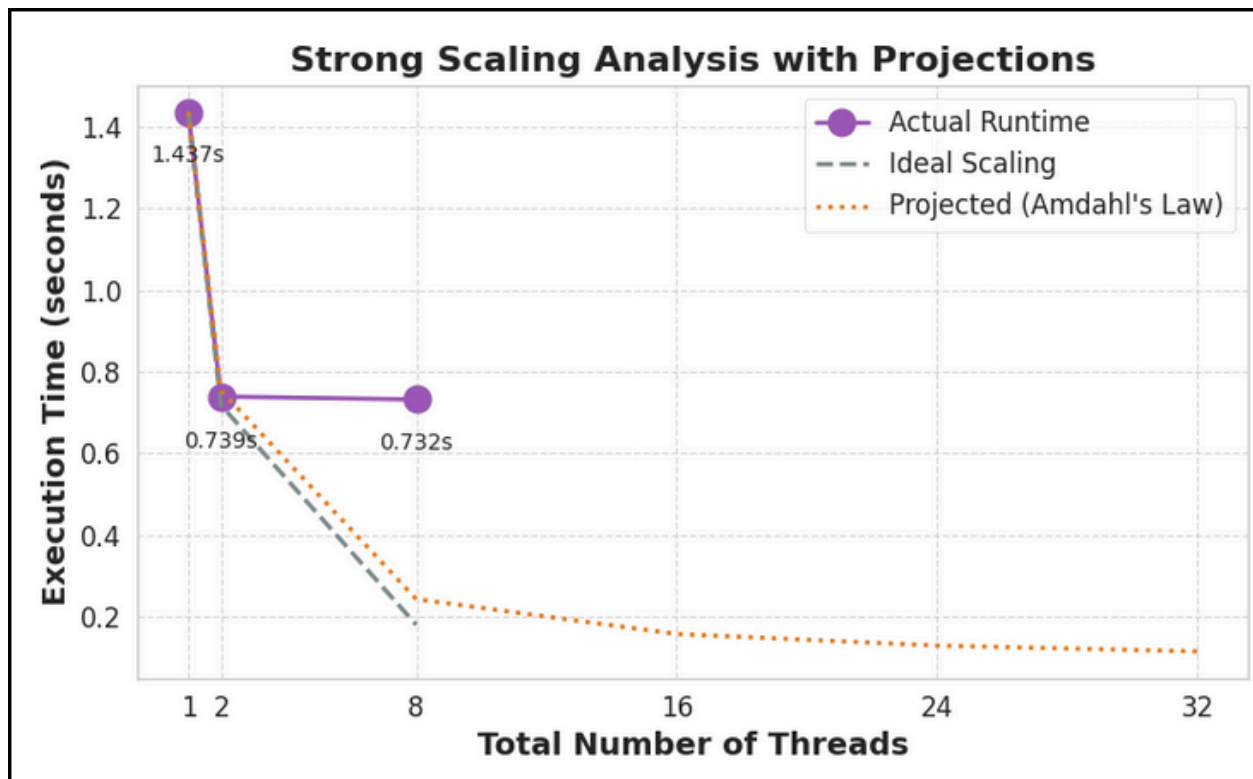


3.5 Strong Scaling Analysis with Projections

Strong scaling tracks execution time as thread count increases (with fixed problem size):

- **1 thread:** 1.437 seconds
- **2 threads:** 0.739 seconds
- **8 threads:** 0.732 seconds

Amdahl's Law projections suggest diminishing gains beyond 32 threads due to serial constraints.

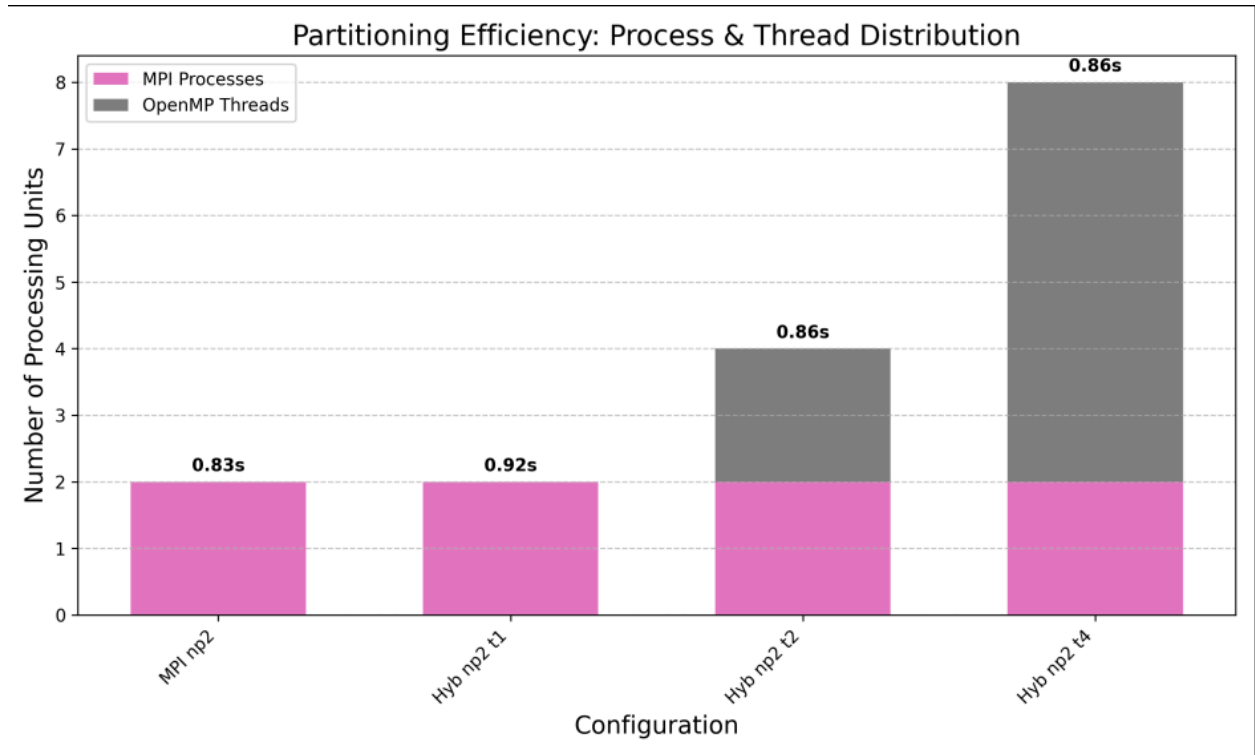


3.6 Partitioning Efficiency

This evaluates workload balance across configurations:

- **MPI np2**: 0.83 seconds
- **Hyb np2 t1**: 0.92 seconds
- **Hyb np2 t2**: 0.86 seconds
- **Hyb np2 t4**: 0.86 seconds

Thread scaling in hybrid setups improves partitioning efficiency and resource utilization.



4. Discussion of Findings

The sequential implementation serves as a baseline, highlighting the compute-intensive nature of processing $10!$ permutations. The MPI-only version halves execution time, benefiting from balanced inter-process work with minimal communication (0.1%). The hybrid version further optimizes runtime by combining process and thread-level parallelism.

The dominance of compute time (99.9–100%) in all configurations favors parallel scalability. Speedup and efficiency analyses show that up to 8 threads yield a $\sim 1.96\times$ speedup, but with decreasing efficiency, as predicted by Amdahl's Law.

Partitioning efficiency suggests optimal load distribution in the hybrid setup. Overall, the hybrid MPI+OpenMP approach slightly outperforms MPI-only due to better utilization of shared memory for compute-bound tasks.

Comparative analysis against theoretical complexity $O(n \cdot n!)$ demonstrates practical runtime reductions, affirming the value of parallel execution.

5. Conclusion

The parallel algorithm for constructing ISTs in bubble-sort networks achieves significant speedups over the sequential approach. The hybrid MPI+OpenMP model delivers the best results, achieving a 49.1% execution time reduction (0.732 seconds using 8 threads), with effective resource balance.

Key findings include:

- **Compute-intensive performance profile**
- **High scalability up to 8 threads**
- **Minimal I/O and communication overhead**
- **Efficient workload distribution**

Future directions include:

- Optimizing serial bottlenecks (e.g., I/O)
- Extending to larger networks like $\mathbf{B}_{12}$

- Applying the approach to generalized bubble-sort or butterfly networks

These enhancements can further unlock the algorithm's potential for practical interconnection network applications.